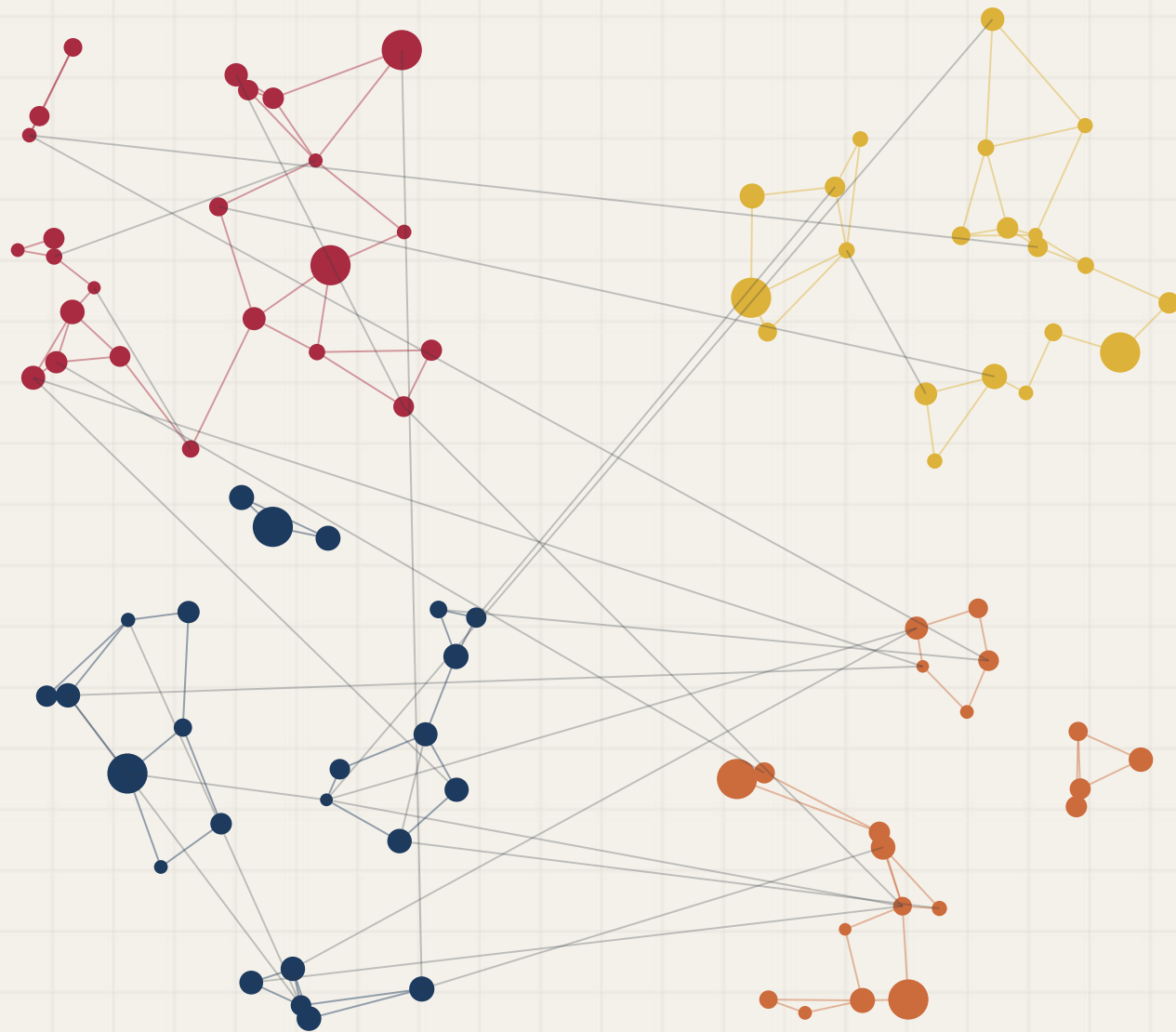




Enquadramento Ideológico

Análise textual de enquadramento ideológico baseada em grafos.

Estrutura de dados 2
Grupo 16
Temática A

Integrantes do Grupo 16



Arthur Fonseca
221031120



Letícia Hladczuk
221039209



Lucas Fujimoto
241025283

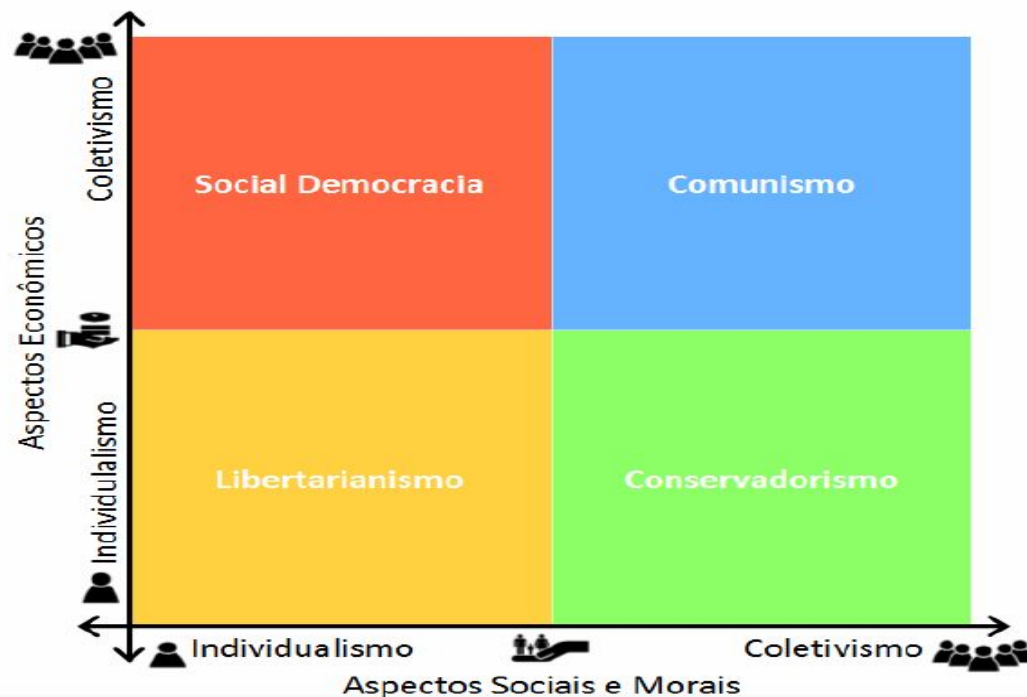


Vitor Feijó
221008516

Descrição do Problema

O problema encontrado pelo nosso grupo foi tentar a partir de um texto não-estruturado, definir o enquadramento ideológico dele. Dessa maneira, o nosso projeto busca responder a pergunta “Dado um texto, qual ideologia ele se enquadra?”

Assim, buscando identificar a ideologia e encaixar em uma das quatro vertentes a seguir:



Modelagem do Grafo

O texto vira um grafo de coocorrência ponderado não-direcionado.
O grafo usa lista de adjacência.

Vértices

Termo lexical extraído do corpus após a remoção de stopwords, pontuação e normalização.

Expressões de mais de uma palavra como `livre_mercado` e `bem_estar_social` são tratadas como um único vértice.

Arestas e pesos

Coocorrência de dois termos na mesma janela de 5 palavras. O peso final de cada aresta é calculado por **NPMI** (Normalized Pointwise Mutual Information), variando de -1 (nunca co-ocorrem) a +1 (sempre juntos).

Modelagem do Grafo

O texto vira um grafo de coocorrência ponderado não-direcionado.
O grafo usa lista de adjacência.

Filtragem

O grafo bruto possui diversas arestas ruidosas. Assim, foi aplicado o algoritmo de **Kruskal** para gerar a árvore geradora máxima, sem formar ciclos. Além disso, foi usado o **Union-Find** para detecção de ciclos.

Comunidades

A detecção de agrupamentos semânticos foi feito com o algoritmo Girvan-Newman, ao identificar grupos de palavras que aparecem relacionadas entre si. Assim, dividindo em comunidades de palavras relacionadas.

Modelagem do Grafo

O texto vira um grafo de coocorrência ponderado não-direcionado.
O grafo usa lista de adjacência.

Ancoragem ideológica

Cada comunidade detectada é atribuída a uma ideologia pela contagem de **seeds** que ela contém. Combinando dois sinais:

- **Seeds léxicas:** termos âncora por ideologia (ex.: “mercado” - âncora libertarianismo).
- **Rótulos supervisionados:** proporção de documentos de cada ideologia no corpus que ativam aquela comunidade.

Coloração e Tamanho dos Vértices

Elemento visual	Significado
Cor do vértice	Ideologia atribuída pela comunidade
Tamanho do vértice	Centralidade de grau no modelo de referência
Espessura da aresta	Número de janelas em que o par co-ocorre no documento
Aresta tracejada	Ponte entre termos de ideologias diferentes
Número na aresta	Co-ocorrências repetidas no documento

Arquitetura do Sistema

O pipeline de processamento opera em três estágios sequenciais. A ingestão de dados sintéticos (Fase 0) alimenta a construção topológica (Fase A), gerando o modelo estático que fundamenta a inferência e a classificação estrutural na ponta final (Fase B).

FASE 0 — Geração do Corpus (generate_corpus.py)

Templates linguísticos + léxicos ideológicos → `corpus.jsonl` (160 documentos rotulados)

FASE A — Treinamento (build_model.py)

`corpus.jsonl`

- `process_document()` × 160
 - ▼ janelas deslizantes (`window_size=5`)
- `build_vocab_from_windows()` + `prune(min_df, max_df)`
 - ▼ Vocabulário filtrado → `count_cooccurrences()`
 - ▼ Grafo bruto → NPML → Kruskal → Girvan-Newman
- `anchor_communities_supervised()`

`model.json`

USO DO
MODELO

FASE B — Inferência (classify.py / app.py)

`texto.txt`

- `process_document()` → janelas do documento
 - ▼ `score_document_graph()` ou `score_document_jaccard()`
 - ▼ **Distribuição Ideológica:**

`{libertarianismo: 75.1%, social-democracia: 14.9%, ...}`

SAÍDAS VISUAIS:

- Terminal: barras ASCII
- PNG: subgrafo estático + distribuição
- HTML: grafo interativo

Geração do Corpus (generate_corpus.py)

O corpus de treino é sintético e rotulado: não usamos dados externos. Ele é gerado por templates linguísticos preenchidos com léxicos ideológicos controlados.

Como funciona?

- 10 templates de frases por ideologia (ex.: "O livre {A} promove {B} e distribui {C} de forma eficiente.")
- Léxico próprio de 18–24 termos por ideologia
- 40 documentos × 4 ideologias = 160 documentos. Cada documento tem 3–5 frases
- Semente aleatória fixa (seed=42) garante reprodutibilidade

Por que sintético?

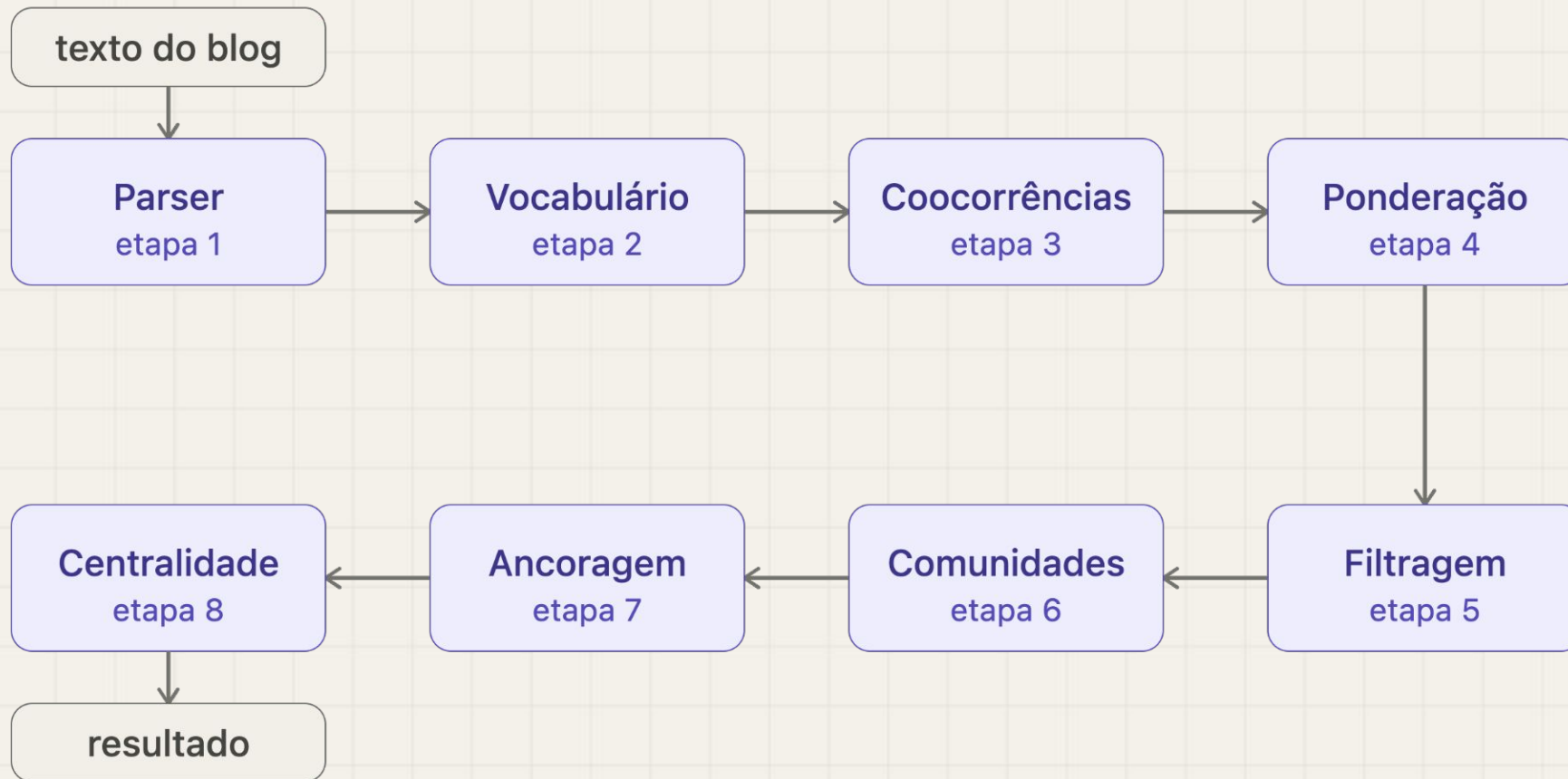
Permite controle total dos rótulos e dos padrões lexicais, essencial para o treino supervisionado e para avaliar o sistema de forma reproduzível. A seção de limitações discute o impacto disso na generalização.

Estrutura de saída — data/raw/corpus.jsonl:

```
{"ideology": "libertarianismo", "text": "O livre comércio promove eficiência e distribui capital de forma eficiente. ..."}  
{"ideology": "conservadorismo", "text": "A família é o alicerce da sociedade e deve ser preservada. ..."}  
...
```


Geração do Modelo (build_model.py)

Executa o pipeline para geração do modelo de referência.



Classificação de um documento (classify.py)

Recebe qualquer arquivo .txt, executa o pipeline de Fase B e gera 4 saídas.

O que acontece internamente:

1. Carrega **model.json** e a Trie de marcadores
2. Roda **process_document()** no texto: tokeniza, remove stopwords, gera janelas deslizantes
3. Chama **classify(windows, model, method=...)**:
 - **graph** (padrão): constrói um grafo de co-ocorrência do próprio documento e combina **node_score** (termos ideológicos presentes) + **edge_score** (pares ideológicos que co-ocorrem na mesma janela). Termos que aparecem juntos no mesmo contexto contribuem mais.
 - **jaccard**: compara apenas o conjunto de termos presentes (bag-of-words ponderado); ignora a estrutura de co-ocorrência do documento.
4. Normaliza os scores para distribuição de probabilidade (soma = 1)
5. Gera as saídas visuais

Diferença entre os métodos:

Método	Considera contexto	Velocidade	Quando usar
graph	Sim	Levemente mais lento	Textos com argumentação estruturada
jaccard	Não	Mais rápido	Textos curtos ou listas de termos

Algoritmos e Conceitos Implementados

Aqui temos os algoritmos utilizados na produção do projeto.

Algoritmo / Conceito	Papel no projeto
Janela deslizante (context window)	Captura co-ocorrências locais entre termos
NPMI (Normalized Pointwise Mutual Information)	Peso estatístico das arestas
Jaccard ponderado	Método alternativo de classificação
Algoritmo de Kruskal	Filtragem do backbone do grafo
Union-Find	Deteção de ciclos para Kruskal
BFS (Busca em Largura)	Base do Brandes + componentes conexos
Betweenness Centrality (algoritmo de Brandes)	Identifica arestas pontes para Girvan-Newman

Algoritmos e Conceitos Implementados

Aqui temos os algoritmos utilizados na produção do projeto.

Algoritmo / Conceito	Papel no projeto
Girvan-Newman	Detecção de comunidades semânticas
Modularidade de Newman	Critério de parada do Girvan-Newman
Propagação de Rótulos	Alternativa mais rápida ao Girvan-Newman
Centralidade de grau	Score de importância dos termos (tamanho do nó)
Trie (árvore de prefixos)	Reconhecimento eficiente de marcadores multipalavra
Disparity Filter	Filtragem alternativa por significância estatística
Ancoragem supervisionada	Combina seeds + rótulos do corpus para atribuir ideologias às comunidades
Grafo co-ocorrência do documento	Scorer da Fase B que captura relações contextuais

Exemplos de Entrada e Saída

O pipeline de inferência transforma textos brutos em diagnósticos estruturados instantaneamente. O modelo extrai as janelas de contexto e calcula a distribuição ideológica projetando o subgrafo do documento contra a base de referência.

Arquivo de Entrada (exemplo.txt)

A privatização de empresas estatais é fundamental para aumentar a eficiência do mercado e atrair capital estrangeiro. O ajuste fiscal e a desregulação do setor produtivo criam condições ideais para o crescimento econômico sustentável. A competitividade da economia depende da liberalização comercial e da redução dos gastos públicos improdutivos. Investimento privado e livre mercado são os motores do desenvolvimento e da produtividade nacional.

python
scripts/classify.py

Saída no Terminal

```
Modelo carregado: 259 termos, 258 arestas
Documento: exemplo.txt (447 caracteres)
Termos únicos no documento: 36
Janelas geradas: 13
```

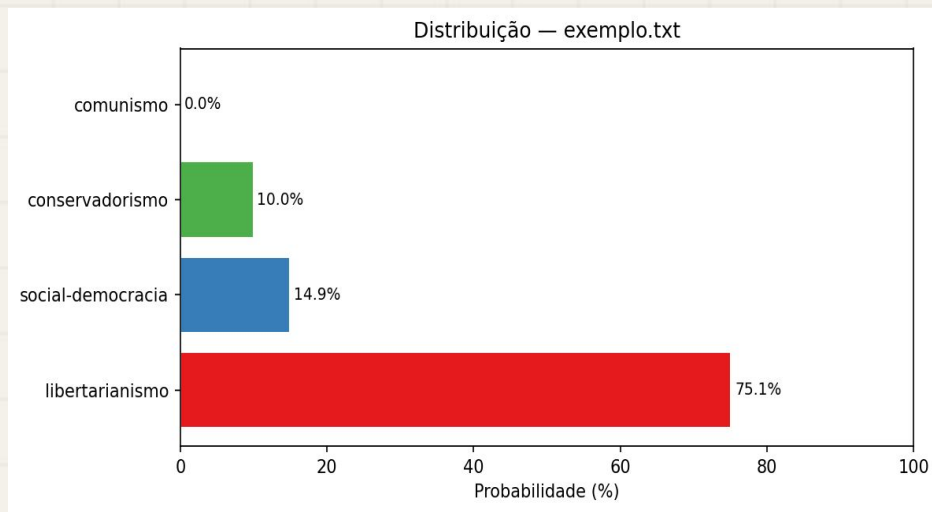
DISTRIBUICAO IDEOLOGICA

libertarianismo	75.1%	#####
social-democracia	14.9%	####
conservadorismo	10.0%	###
comunismo	0.0%	

```
Enquadramento dominante: libertarianismo (75.1%)
```

Exemplos de Saída: Análise de Resultados

Distribuição Ideológica (Gráfico de Barras)

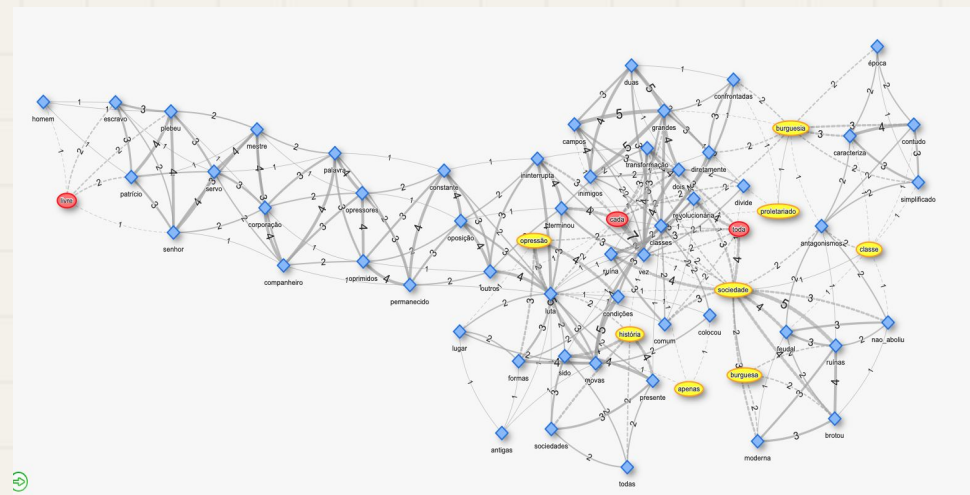


Diagnóstico Percentual

Apresenta a distribuição de probabilidade normalizada do documento.

O enquadramento dominante é definido pela combinação estrutural dos termos presentes e suas fortes coocorrências locais.

Grafo Interativo (HTML)



Exploração Topológica (Gerada via HTML)

A visualização permite inspecionar a semântica interna do texto:

- **Tamanho dos nós:** Reflete a centralidade de grau do termo no modelo de referência.
- **Espessura das arestas:** Indica o volume de coocorrências na mesma janela de contexto.
- **Arestas tracejadas:** Revelam pontes onde o texto conecta vocabulários de ideologias diferentes.

Exemplos de Saída: Análise de Resultados

Interface Streamlit

Configurações

Método de classificação ?

☒ Janela de contexto

☐ Jaccard

Modelo carregado

Termos no vocabulário

259

Ideologias

4

☒ libertarianismo

☒ conservadorismo

☒ comunismo

Deploy



Enquadramento Ideológico

Classifica textos políticos usando grafos de co-ocorrência.

Cole ou digite o texto para análise:

A privatização de empresas estatais é fundamental para aumentar a eficiência do mercado e atrair capital estrangeiro. O ajuste fiscal e a desregulação do setor produtivo criam condições ideais para o crescimento econômico sustentável. A competitividade da economia depende da liberalização comercial e da redução dos gastos públicos improdutivos. Investimento privado e livre mercado são os motores do desenvolvimento e da produtividade nacional.

Analisar

63 palavras

Enquadramento dominante: **LIBERTARIANISMO (75.1%)**

Termos únicos no texto: 36 · Janelas geradas: 13 · Método: graph

Distribuição ideológica

Testes

Tests coverage

```
===== tests coverage =====
coverage: platform darwin, python 3.11.9-final-0

Name                               Stmts  Miss  Cover   Missing
-----
src/__init__.py                     0      0  100%
src/analysis/__init__.py            0      0  100%
src/analysis/centrality.py          53      0  100%
src/analysis/communities.py        105      6   94%   35, 144, 148, 166-167, 206
src/analysis/filtering.py           44      2   95%   86, 90
src/analysis/traversal.py           25      0  100%
src/config.py                       12      0  100%
src/datastructures/__init__.py       0      0  100%
src/datastructures/graph.py          59      1   98%   199
src/datastructures/trie.py           40      0  100%
src/datastructures/union_find.py     36      1   97%    65
src/graph_build/__init__.py          0      0  100%
src/graph_build/cooccurrence.py      29      0  100%
src/graph_build/vocabulary.py        47      0  100%
src/graph_build/weighting.py         54      1   98%    71
src/model/__init__.py               0      0  100%
src/model/anchoring.py              81     41   49%   66-89, 118-159
src/model/reference_model.py         53     53    0%   3-188
src/parser/__init__.py               0      0  100%
src/parser/lemmatize.py             44     33   25%   16-24, 34-38, 56-66, 84-93
src/parser/pipeline.py              73      2   97%   142, 151
src/parser/sanitize.py              17      0  100%
src/parser/stopwords.py              6      0  100%
src/parser/windows.py               17      0  100%
src/scoring/__init__.py              0      0  100%
src/scoring/classifier.py           45      0  100%
src/scoring/doc_graph.py            11      0  100%
src/viz/__init__.py                 0      0  100%
src/viz/render.py                   131    131    0%   3-255
src/viz/render_interactive.py        63     63    0%   3-207
TOTAL                               1045    334   68%

===== 216 passed in 0.39s =====
```

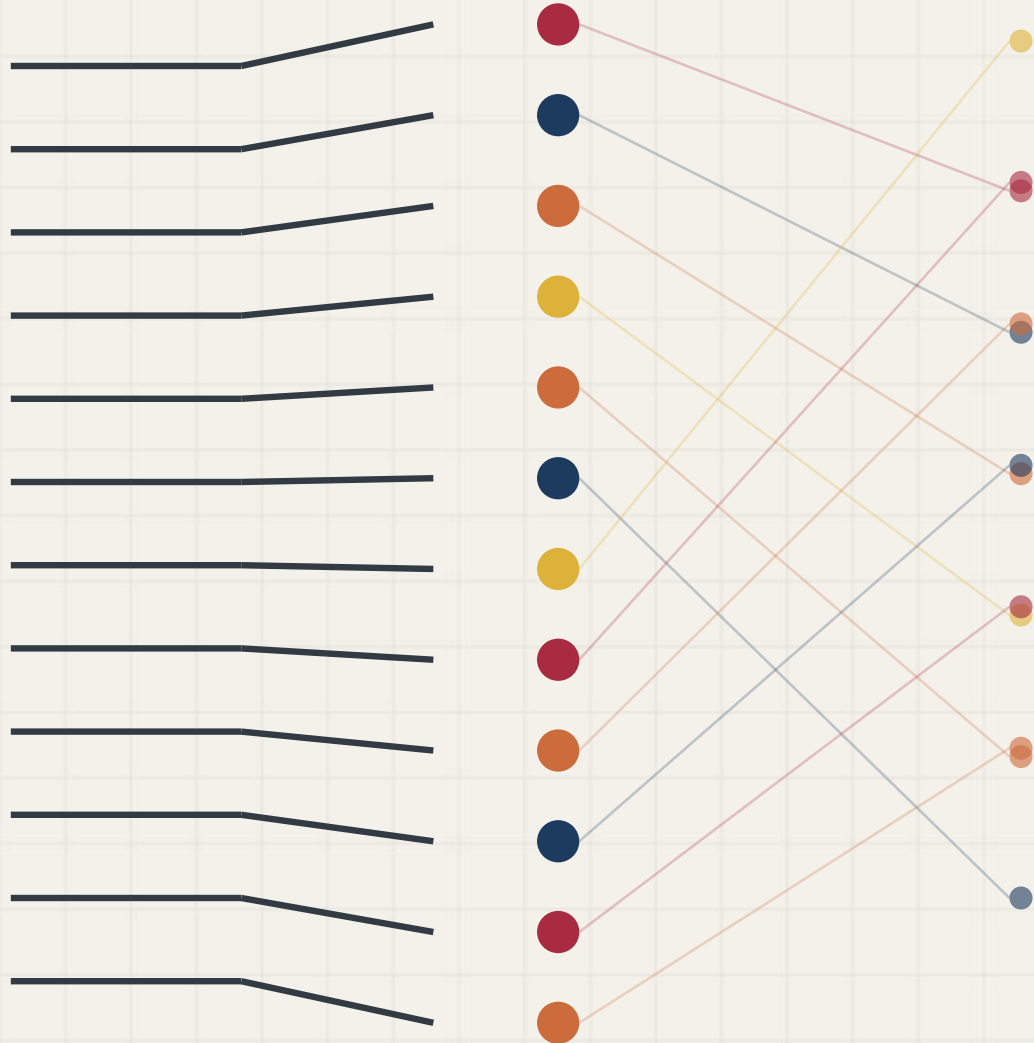
216 testes distribuídos em 7 módulos:

Módulo de teste	O que cobre
test_datastructures.py	Graph , Trie , UnionFind
test_parser.py	tokenização, stopwords, janelas
test_graph_build.py	vocabulário, coocorrências, ponderação
test_analysis.py	filtragem, comunidades, centralidade, BFS
test_anchoring.py	seeds, ancoragem supervisionada
test_scoring.py	Jaccard, grafo do documento, normalização
test_integration.py	pipeline ponta-a-ponta

Uso de LLM no Desenvolvimento

A inteligência artificial foi empregada para agilizar a preparação da pipeline de dados:

- **Dados Sintéticos:** Expansão dos templates linguísticos para escalar a criação do corpus de treinamento (`corpus.jsonl`), garantindo diversidade de exemplos e controle dos rótulos.
- **Enriquecimento Lexical:** Assistência na descoberta de sinônimos e mapeamento de expressões multipalavra, fortalecendo a precisão dos nossos dicionários de ancoragem (`seeds.json` e `markers.txt`).
- **Higienização e Parser:** Geração e validação ágil de expressões regulares para otimizar a limpeza bruta dos textos, entregando tokens sem ruído para o algoritmo de janelas deslizantes.



Obrigado pela atenção!